

Django REST Framework (DRF)

Building scalable web APIs using DRF views, routers, serializers, validation, and renderers.

1. What is Django REST Framework?

Django REST Framework (DRF) is a powerful, flexible toolkit built on top of Django that simplifies creating RESTful Web APIs. It provides browsable APIs, serialization, authentication policies, input validation, and generic view controllers out of the box.

2. Installation & Configuration

```
# Install DRF via pip or pipenv pip install djangorestframework # settings.py
INSTALLED_APPS = [ ... 'rest_framework', # Enable Django REST Framework 'debug_toolbar', #
Optional: For performance debugging & SQL profiling ]
```

3. Views in DRF: FBVs vs. CBVs vs. ViewSets

View Type	Construct	Best Use Case
Function-Based Views	<code>@api_view(['GET', 'POST'])</code>	Simple endpoints, custom logic, rapid prototyping.
Class-Based Views (CBVs)	Inherit <code>APIView</code>	Explicit separation of HTTP methods (<code>get()</code> , <code>post()</code>).
Generic Views	<code>ListCreateAPIView</code> , etc.	Standard CRUD endpoints with minimal code.
ViewSets	<code>ModelViewSet</code>	Full CRUD suites automatically mapped to routers.

4. Routing in DRF

ViewSets rely on DRF **Routers** to generate URL patterns automatically according to standard REST conventions.

```
# urls.py from rest_framework.routers import DefaultRouter from .views import
MenuItemViewSet router = DefaultRouter() router.register(r'menu-items', MenuItemViewSet,
basename='menu-item') urlpatterns = router.urls
```

5. Serialization & Deserialization

Serialization converts complex data types (such as Django QuerySets or model instances) into native Python datatypes that can easily be rendered into **JSON** or XML. **Deserialization** converts parsed incoming JSON payload back into complex types after validating incoming data.

Serializer Class

Explicitly defines field definitions and custom data transformation logic similar to Django Forms.

ModelSerializer Class

Shortcut class that automatically creates fields matching a Django Model's schema and generates default create/update implementations.

6. Model & Relationship Serializers

```
# serializers.py from rest_framework import serializers from .models import Category, MenuItem class CategorySerializer(serializers.ModelSerializer): class Meta: model = Category fields = ['id', 'title'] class MenuItemSerializer(serializers.ModelSerializer): # Relationship Serialization: Nested, String, or PrimaryKey category = CategorySerializer(read_only=True) category_id = serializers.IntegerField(write_only=True) class Meta: model = MenuItem fields = ['id', 'title', 'price', 'inventory', 'category', 'category_id']
```

7. Deserialization & Data Validation

Incoming payloads are processed using `serializer.is_valid()`. DRF supports field-level validation, object-level validation, and custom validators.

```
class MenuItemSerializer(serializers.ModelSerializer): price = serializers.DecimalField(max_digits=6, decimal_places=2) # Field-level validation rule (validate_) def validate_price(self, value): if value < 0: raise serializers.ValidationError("Price cannot be negative.") return value # Object-level validation rule across multiple fields def validate(self, attrs): if attrs['title'].lower() in attrs['description'].lower(): raise serializers.ValidationError("Title should not repeat in description.") return attrs
```

8. DRF Renderers

Renderers handle converting internal Python primitives into formatted client responses based on content negotiation headers (`Accept` header).

Renderer Class	Content Type / Output	Description
<code>JSONRenderer</code>	<code>application/json</code>	Standard production renderer for client applications and APIs.
<code>BrowsableAPIRenderer</code>	<code>text/html</code>	Renders an interactive HTML web interface for testing and viewing APIs.
<code>XMLRenderer</code>	<code>application/xml</code>	Optional renderer provided by add-on packages for XML payloads.

9. Django Debug Toolbar

Optimizing API Queries with Debug Toolbar

The **Django Debug Toolbar** helps identify performance bottlenecks, duplicate SQL queries, and N+1 query problems in API views.

Key Solutions:

- Use `select_related()` for foreign key / one-to-one relationships to perform single SQL joins.
- Use `prefetch_related()` for many-to-many and reverse foreign key lookups.

10. Complete DRF ModelViewSet & Router Pattern

```
# views.py
from rest_framework import viewsets
from .models import MenuItem
from .serializers import MenuItemSerializer

class MenuItemViewSet(viewsets.ModelViewSet):
    queryset = MenuItem.objects.select_related('category').all()
    serializer_class = MenuItemSerializer
```

Module Key Takeaways

- `ModelViewSet` replaces all CRUD views with a single concise class definition.
- Use `ModelSerializer` to automatically map database models to JSON APIs.
- Always implement explicit field and object validators to prevent dirty data entry.
- Audit API SQL query efficiency using the Django Debug Toolbar.